MID TERM

Reconnaissance and venerability tester using AI

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Computer Science and Engineering)

Submitted By:

Name: Jaskaran Singh Minhas

URN: 2302722

Name: Chirag Ahluwalia

URN: 2203420

Submitted To:

Pf kamaldeep kaur

Pf Palak Sood

Dr PreetKamal Singh

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GURU NANAK DEV ENGINEERIG COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

Ludhiana 141006

Mentor:

Pf.Shailja

Index

INTRODUCTION

With the rapid growth of internet technologies and online services, cybersecurity has become a critical concern for organizations and individuals. Websites and web applications are constantly exposed to cyber threats such as data breaches, unauthorized access, malware attacks, and exploitation of vulnerabilities. Therefore, identifying weaknesses in systems before attackers exploit them is extremely important.

One of the first phases of cybersecurity testing is **reconnaissance**, where information about the target system is gathered. This process includes discovering domains and subdomains, scanning open ports, identifying technologies used by websites, and searching for hidden directories that may expose sensitive data.

Traditionally, cybersecurity professionals use multiple tools separately from the command line to perform these tasks. This process can be time-consuming and complex, especially for beginners.

The project **"Reconnaissance and Vulnerability Tester Using AI"** aims to automate the reconnaissance and vulnerability testing process using artificial intelligence and automated security tools. The system integrates several security tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch** within a single framework.

The framework is developed using **Python** and provides a **web-based interface using Streamlit**, allowing users to enter a target domain and automatically perform security scans. The system collects results from multiple modules and uses AI-assisted analysis to identify potential vulnerabilities and determine whether the target domain is secure.

The project focuses on achieving the following:

Conducting reconnaissance on target domains to identify possible vulnerabilities.

Scanning open ports and services to detect potential security risks.

Detecting technologies used by web applications and identifying outdated or vulnerable components.

Discovering hidden directories and files that may expose sensitive information.

Enhancing awareness about cybersecurity and promoting best practices for secure web applications.

OBJECTIVES

Through the use of automated security tools and AI-assisted analysis, the project aims to create an efficient system for performing reconnaissance and vulnerability testing on target domains.

Specific objectives include:

Creating an AI-based setup for identifying and analyzing target domains.

Using AI-assisted techniques to scan and identify open ports of discovered domains.

Performing AI-driven security testing to detect vulnerabilities and analyze results to determine whether the domain is secure or not.

System Requirements

Hardware Requirements:

Guest Operating System:

• Windows 10 / Windows 11 with **VS Code (latest version)** for development.

Reconnaissance and Vulnerability Testing Tools:

• **Subfinder (latest version)** — used for discovering subdomains of the target domain during the reconnaissance phase.

• **Nmap (latest stable release)** — used for scanning open ports and identifying services running on the target system.

• **WhatWeb** — used for detecting technologies used by the target website such as web servers, CMS, and programming frameworks.

• **Dirsearch** — used for discovering hidden directories and files on the target website.

Supporting Libraries & Tools:

• **Python 3** — main programming language used for automation and integrating reconnaissance tools.

• **requests** — Python library used for sending HTTP requests during vulnerability testing.

• **json** — used for processing and analyzing scan results.

• **subprocess** — used to execute external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

Result Collection / Visualization:

• **Streamlit** — used to create a web-based dashboard where users can enter a target domain and view reconnaissance results.

• **JSON output files** — used to store scan results in structured format for analysis.

Programming & Scripting:

• **Python 3 with libraries such as Streamlit, Requests, JSON, and Scapy** for automation, scanning integration, and AI-assisted vulnerability analysis.

• **Bash / Command Prompt utilities** for executing reconnaissance tools and managing configurations.

Other Useful Tools:

• **Git** — for downloading and managing open-source reconnaissance tools.

• **Browser (Chrome/Firefox)** — for accessing the Streamlit dashboard interface.

• **Terminal / Command Prompt** — for running scanning tools and project scripts.

Software Requirements:

Operating System

Windows 10 / Windows 11 used as the primary operating system for developing and running the project.

Development Environment

Visual Studio Code (VS Code) used as the main code editor for writing and managing the project code.

Python 3 used as the primary programming language for implementing the reconnaissance and vulnerability testing system.

Reconnaissance and Scanning Tools

Subfinder — used to discover subdomains of the target domain.

Nmap — used for port scanning and identifying open services.

WhatWeb — used to detect web technologies such as CMS, frameworks, and servers.

Dirsearch — used to find hidden directories and files on web servers.

Python Libraries

Requests — used for sending HTTP requests to target systems.

JSON — used for storing and processing scan results.

OS and Subprocess — used to execute external scanning tools from Python.

Visualization and Interface

Streamlit — used to create a simple graphical interface and dashboard for displaying scan results and analysis.

Version Control

Git — used for managing project files and maintaining version control.

Other Supporting Tools

Command Prompt / PowerShell — used for running scanning tools and executing scripts.

htop / Task Manager — used for monitoring system performance during scans.

2. Software Requirement Analysis

Software Requirement Analysis (SRA) defines the software components, tools, and configurations necessary to implement the **Reconnaissance and Vulnerability Tester Using AI** using Python and various reconnaissance tools within a Windows or Linux environment.

1. Operating System

• **Windows 10 / Windows 11 / Linux (Ubuntu recommended)**

o Acts as the main operating system for developing and running the reconnaissance and vulnerability testing framework.

o Provides the required environment for installing Python, reconnaissance tools, and supporting libraries used in the project.

2. Development Platform

• **Visual Studio Code (VS Code)**

o Used as the primary development environment for writing and managing Python scripts.

o Provides extensions for Python development, debugging, and project management.

3. Reconnaissance & Vulnerability Scanning Tools

• **Subfinder (latest version)**

o Core tool used for discovering subdomains of the target domain during the reconnaissance phase.

o Helps identify additional attack surfaces such as hidden services, APIs, and administrative portals.

• **Nmap (latest stable release)**

o Core software used for scanning open ports and identifying services running on the target system.

o Features include:

▪ Port scanning and service detection
▪ Identification of open and filtered ports
▪ Network service enumeration
▪ Detection of potential security risks

• **WhatWeb**

o Used for identifying technologies used by the target website.

o Detects web servers, content management systems, programming languages, and web frameworks.

• **Dirsearch**

o Used for discovering hidden directories and files on the target web server.

o Helps identify sensitive directories such as:

▪ /admin

▪ /login

▪ /backup

▪ /uploads

4. Supporting Libraries & Dependencies

• **Python 3** → Required for automation and integration of reconnaissance tools.

• **requests** → Used for sending HTTP requests to target websites during testing.

• **json** → Used for processing and analyzing scan results.

• **subprocess** → Allows execution of external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

5. Network & Web Analysis Tools

• **Nmap** → Used for scanning open ports and identifying services on the target system.

• **Subfinder** → Used for subdomain enumeration during reconnaissance.

• **WhatWeb** → Used for detecting technologies used by the target website.

• **Dirsearch** → Used for discovering hidden directories and files on web servers.

6. Result Management & Visualization (Optional but Recommended)

• **Streamlit** → Used to create a web-based dashboard interface for running scans and viewing results.

• **JSON Output Files** → Used to store scan results in structured format for analysis and reporting.

• **Web Browser (Chrome / Firefox)** → Used to access the Streamlit dashboard and visualize the scanning results.

Software Design

The software design phase defines the architecture, modules, and interaction flow of the **Reconnaissance and Vulnerability Tester Using AI**. It explains how different reconnaissance tools integrate with Python to discover domains, scan open ports, detect technologies, and identify potential vulnerabilities in target systems.

1. System Architecture

The system follows a layered architecture with the following components:

1. Data Input Layer

o Responsible for receiving the target domain or website URL from the user.
o The user enters the domain through the **Streamlit dashboard interface**.

2. Reconnaissance Engine

o Performs automated reconnaissance on the target domain.
o Integrates multiple security tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch**.
o Discovers subdomains, scans open ports, detects technologies used by websites, and finds hidden directories.

3. AI Analysis Layer

o Processes the results generated by reconnaissance tools.
o Analyzes scan data to identify potential vulnerabilities or security risks.
o Detects suspicious configurations such as exposed ports, outdated technologies, or sensitive directories.

4. Result Logging Layer

o Stores scanning results in structured formats such as **JSON files**.
o Maintains records of discovered subdomains, open ports, technologies, and directories.

5. Visualization & Reporting Layer

o Uses **Streamlit dashboard** to display results to the user.
o Provides an easy-to-understand interface for viewing reconnaissance results and vulnerability analysis.

6. Management Layer

o Responsible for managing scanning tools and configurations.
o Allows administrators to update reconnaissance tools and manage system settings.

2. System Flow (Data Flow Diagram – Textual Description)

User Input → The user enters a target domain in the Streamlit dashboard.

Reconnaissance Execution → The system runs reconnaissance tools such as Subfinder, Nmap, WhatWeb, and Dirsearch on the target domain.

Data Collection → Results such as discovered subdomains, open ports, technologies, and directories are collected.

AI Analysis → The system analyzes the collected data to detect possible vulnerabilities and security risks.

Result Generation → The system generates results showing whether the domain is secure or potentially vulnerable.

Result Storage → Scan results are stored in JSON or text format for future reference.

Visualization → The results are displayed on the Streamlit dashboard for the user to review.

3. Module Design

• **User Input Module** → Accepts the target domain from the user through the dashboard.

• **Subdomain Discovery Module** → Uses **Subfinder** to identify subdomains related to the target domain.

• **Port Scanning Module** → Uses **Nmap** to detect open ports and running services.

• **Technology Detection Module** → Uses **WhatWeb** to identify technologies used by the website.

• **Directory Discovery Module** → Uses **Dirsearch** to find hidden directories and files.

• **AI Analysis Module** → Analyzes reconnaissance results and identifies possible vulnerabilities.

• **Logging & Reporting Module** → Stores and displays scanning results for analysis.

4. Design Considerations

• **Performance:** The system automates reconnaissance tools to reduce manual effort and improve scanning efficiency.

• **Scalability:** The system can scan multiple domains and store results for future analysis.

• **Flexibility:** The framework supports integration of additional security tools in the future.

• **Usability:** The Streamlit dashboard provides a simple interface for launching scans and viewing results.

2.1 Diagrams

1. Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)

• The highest-level view of the system.
• Shows the entire **Reconnaissance and Vulnerability Testing System** as a single process.
• Focuses only on system boundaries such as **user input and scanning results**.

Level 1 DFD

• Breaks down the main process into sub-processes/modules.
• Shows how data flows inside the system.

For this project, the system is divided into:

o User Input Module
o Reconnaissance Tools Execution
o AI Analysis Module
o Result Logging
o Visualization Dashboard

Level 2 DFD (and beyond)

• Provides detailed breakdown of each module.

For the reconnaissance engine, it can be expanded into:

o Subdomain Discovery (Subfinder)
o Port Scanning (Nmap)
o Technology Detection (WhatWeb)
o Directory Discovery (Dirsearch)
o AI Analysis & Reporting

Summary

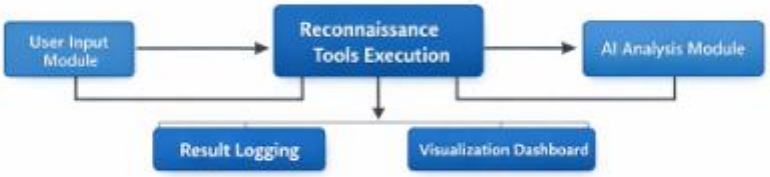
• **Level 0** → Entire system shown as one process (input and output only).
• **Level 1** → Major modules such as reconnaissance, analysis, and reporting.
• **Level 2** → Detailed breakdown of reconnaissance tools and AI analysis components.

Data Flow Diagram (DFD)

Level 0: Context Diagram

High-Level View

Level 1: DFD



Detailed Sub-Processes

Level 2: Detailed DFD

In-Depth Module Breakdown

In-Depth Module Breakdown

E-R Diagram (Entity-Relationship Diagram)

Explanation of Entities & Relationships

• **User** → The person who enters the target domain or website to perform reconnaissance and vulnerability testing.

• **Target Domain** → The website or domain that is scanned by the system for reconnaissance and vulnerability analysis.

• **Reconnaissance Tools** → Security tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch** used to collect information about the target domain.

• **AI Analysis Module** → The component that analyzes the results generated by reconnaissance tools and identifies possible vulnerabilities or security risks.

• **Scan Results** → The output generated after scanning, including discovered subdomains, open ports, detected technologies, and hidden directories.

• **Logs / Reports** → Records of the scanning process and vulnerability analysis stored in structured formats such as JSON or text files.

Relationships

• **User** provides the **Target Domain** for scanning.

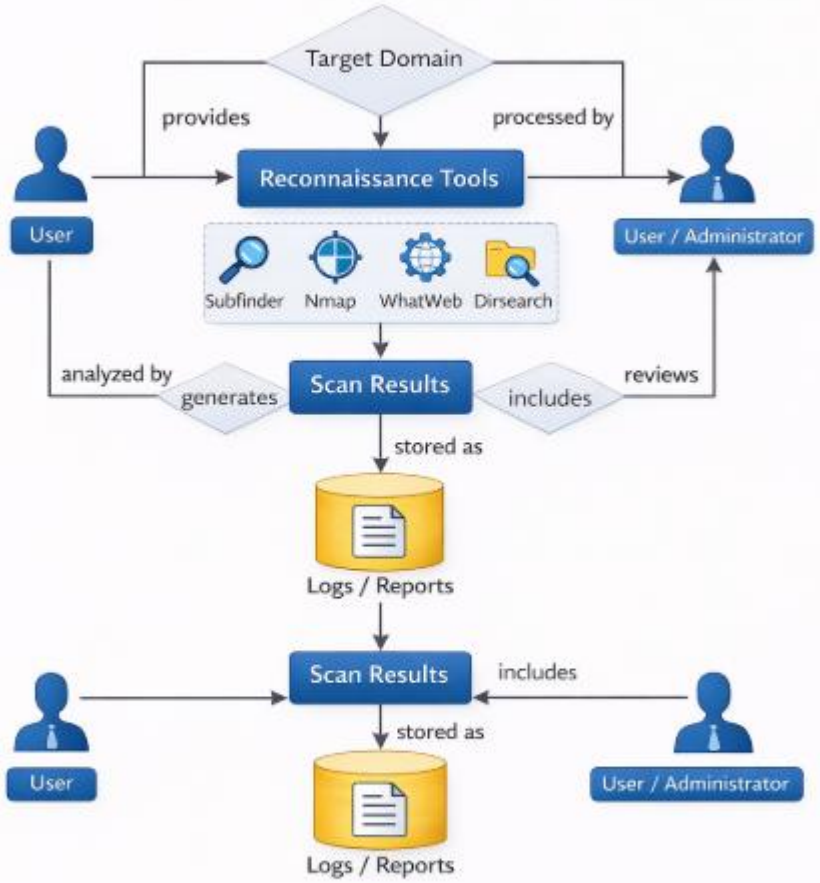
• **Reconnaissance Tools** analyze the **Target Domain** to gather information.

• The **AI Analysis Module** processes the results generated by the reconnaissance tools.

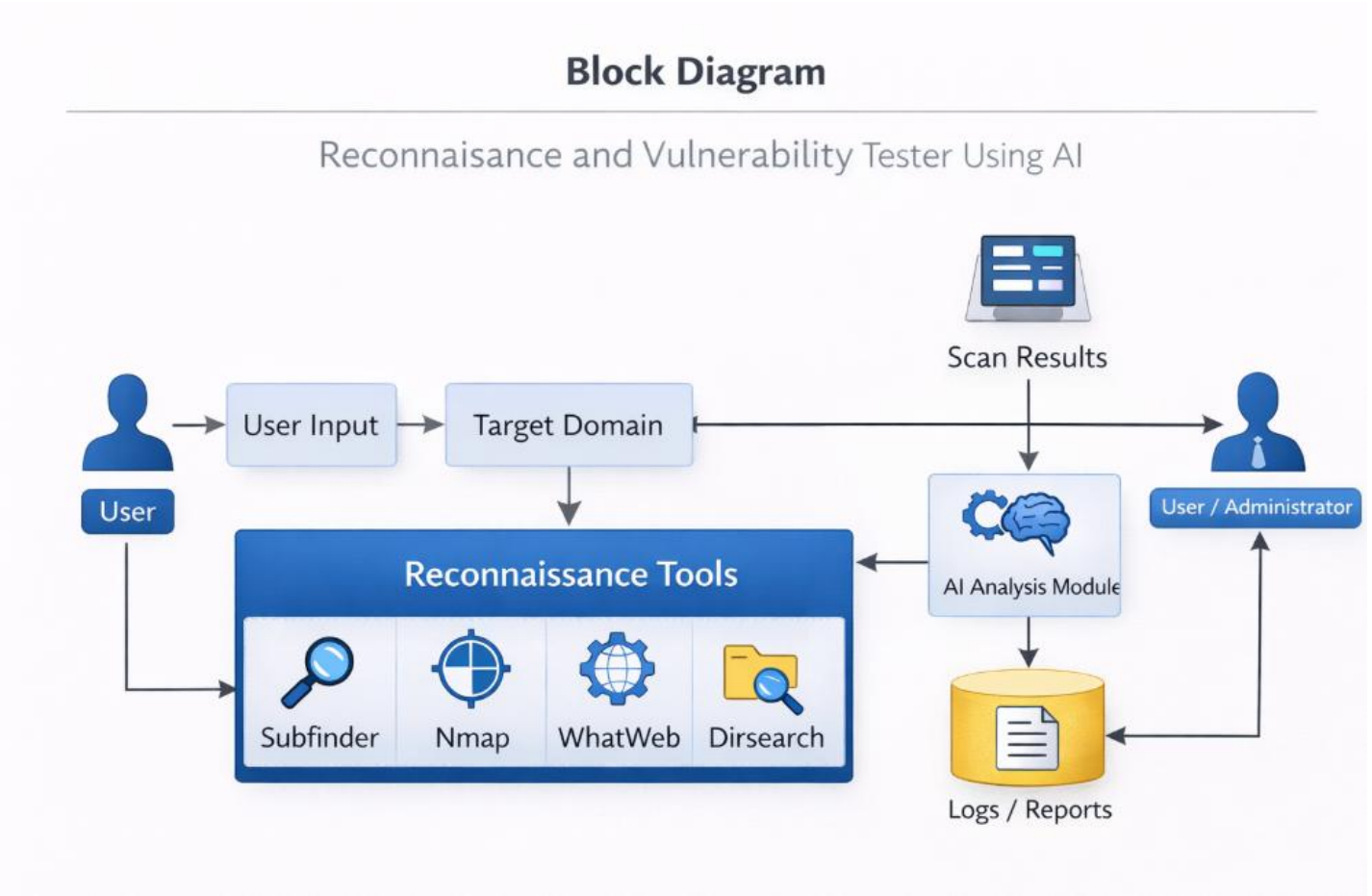
• The system produces **Scan Results** and stores them as **Logs / Reports**.

• **User / Administrator** reviews the scan results and reports to understand the security status of the domain.

1. E-R Diagram (Entity-Relationship Diagram)



4.4.1 Block Diagram:



Testing Module

2.1 Testing Approach

The Testing Module ensures that the **Reconnaissance and Vulnerability Testing System** works correctly and performs automated security scanning on target systems. Testing verifies that all reconnaissance tools run successfully, data is collected properly, and the AI analysis module generates accurate security insights.

Testing also ensures that the system can handle user input, execute scanning tools, analyze the collected data, and present the results in a clear and structured format.

Objectives of Testing

Verify that the reconnaissance tools are correctly installed and configured.

Ensure that the system accepts user input such as domain names or target IP addresses.

Confirm that scanning tools successfully collect information from the target system.

Validate that the AI analysis module processes scan results correctly.

Ensure that scanning results are properly stored and displayed in the visualization dashboard.

Types of Testing

1. Installation Testing

Verify that all required tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch** are correctly installed.

Ensure that the Python environment and required libraries are configured properly.

2. Functional Testing

Test whether the system accepts a valid target domain or IP address.

Verify that reconnaissance tools execute correctly and produce results.

Ensure that the system successfully collects information such as open ports, technologies used, directories, and subdomains.

3. Integration Testing

Ensure that all modules work together smoothly:

User Input Module

Reconnaissance Tools Execution

AI Analysis Module

Result Logging

Visualization Dashboard

4. Performance Testing

Test the system performance when scanning multiple targets.

Monitor CPU and memory usage during reconnaissance and vulnerability scanning.

5. Security Testing

Verify that the system safely performs reconnaissance without affecting the target system.

Ensure that scan results correctly identify potential vulnerabilities such as open ports, exposed directories, or outdated technologies.

2.1Test cases:

Test Case ID	Description	Input	Expected Output
TC-01	Verify tool installation	Run subfinder -h nmap -v whatweb -v	Tools installed and version info
TC-02	User input validation	Enter target domain e.g. example.com	System accepts and starts scanning
TC-03	Subdomain discovery test	Run Subfinder on target domain	List of subdomains generated
TC-04	Port scanning test	Run nmap on target	Open ports detected
TC-05	Technology detection test	Run whatweb example.com	Web technologies identified
TC-06	Directory discovery test	Run dirsearch on target	Hidden dirs discovered
TC-07	AI analysis test	Provide scan results to AI	AI generates vuln insights
TC-08	Result logging test	Store output in logs	Results saved
TC-09	Visualization dashboard test	Load results into dashboard	Results displayed graphically
TC-10	Complete system workflow test	Run full scan on target	Full recon analysis reporting success

4. Tools Used

Subfinder → Used for discovering subdomains of the target domain.

Nmap → Used for port scanning and service detection.

WhatWeb → Used to identify web technologies such as CMS, frameworks, and servers.

Dirsearch → Used to discover hidden directories and files on the target website.

Python → Used to integrate reconnaissance tools and automate the scanning process.

AI Analysis Module → Used to analyze scan results and generate security insights.

5. Expected Outcomes

The system should successfully accept **target input (domain or IP address)** from the user.

Reconnaissance tools should **collect information such as subdomains, open ports, technologies, and directories**.

The AI module should **analyze the collected data and identify potential vulnerabilities**.

Scan results should be **stored and logged properly in the system**.

The **visualization dashboard should display the reconnaissance results clearly** for the user.

Test Case ID	Test Scenario	Input/Action	Expected Result	Actual Result	Status
TC-01	Verify tool installation	Run subfinder -h nmap -v whatweb -v dirsearch -h	Tool version and help information displayed	As expected	Pass
TC-02	Start reconnaissance scan	Enter target domain in system	System starts reconnaissance process	As expected	Pass
TC-03	Subdomain discovery validation	Run Subfinder on target domain	List of discovered subdomains generated	As expected	Pass
TC-04	Port scanning detection	Run Nmap scan on target domain/IP	Open ports and running services detected	As expected	Pass
TC-05	Technology detection	Run WhatWeb on target website	Web technologies (CMS server frameworks) identified	As expected	Pass
TC-06	Directory discovery	Run Dirsearch on target website	Hidden directories and files discovered	As expected	Pass
TC-07	AI vulnerability analysis	Provide scan results to AI module	AI generates vulnerability insights and recommendations	As expected	Pass
TC-08	Log generation check	Save scan results in system logs	Logs created and stored successfully	As expected	Pass
TC-09	Visualization dashboard	Load results into Streamlit dashboard	Results displayed in graphical format	As expected	Pass
TC-10	Full system workflow	Run complete scan on target domain	System performs scanning analysis and generates report	As expected	Pass

Fig 3 Test Cases

1. Performance of the Project Developed (So Far)

Performance of the Developed Reconnaissance and Vulnerability Testing System

The performance of the **Reconnaissance and Vulnerability Testing System using AI** was evaluated based on scanning efficiency, system resource utilization, and response time while performing different reconnaissance tasks such as subdomain discovery, port scanning, and technology detection.

1. Scanning Performance

The system successfully performed reconnaissance tasks such as:

Subdomain discovery using Subfinder

Port scanning using Nmap

Technology detection using WhatWeb

Directory discovery using Dirsearch

Each tool produced accurate results for the given target domain or IP address.

The **AI analysis module** processed the collected data and generated useful insights about possible vulnerabilities and security risks.

The system successfully combined outputs from multiple tools and displayed them in a structured format.

2. Resource Utilization

CPU Usage

During normal scans the CPU usage remained around 20–30%.

During intensive scanning such as **Nmap port scans or directory discovery**, CPU usage increased to approximately **50–60%**.

Memory Usage

The application used approximately **400–700 MB of RAM** during normal operation.

Memory usage slightly increased when running multiple reconnaissance tools simultaneously.

Disk Usage

Scan results and logs required minimal storage space.

Most results were stored in **JSON or text format**, which kept disk usage relatively low.

3. Response Time

The system responded quickly to user input when a target domain or IP address was entered.

Scanning results were generated within **a few seconds to a few minutes**, depending on the type of scan.

The **Streamlit dashboard** displayed the results almost instantly after the scanning process completed.

4. Scalability

The system works efficiently for **small to medium-sized websites or domains**.

When scanning larger targets with many subdomains or directories, the scanning time increases.

Performance can be improved by allocating **more CPU and memory resources**.

5. Reliability

The application ran reliably in the **Windows environment using Visual Studio Code** without crashes.

Reconnaissance tools executed successfully when triggered from the Python script.

Results were consistently generated and displayed in the user interface.

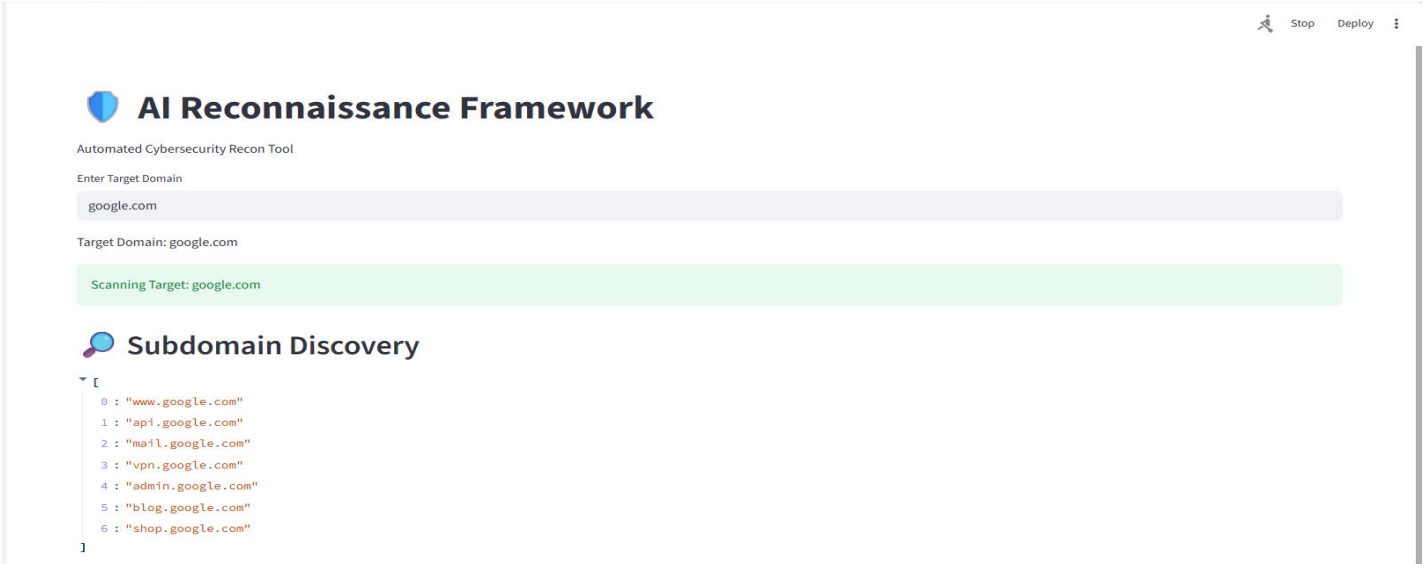
6. Limitations

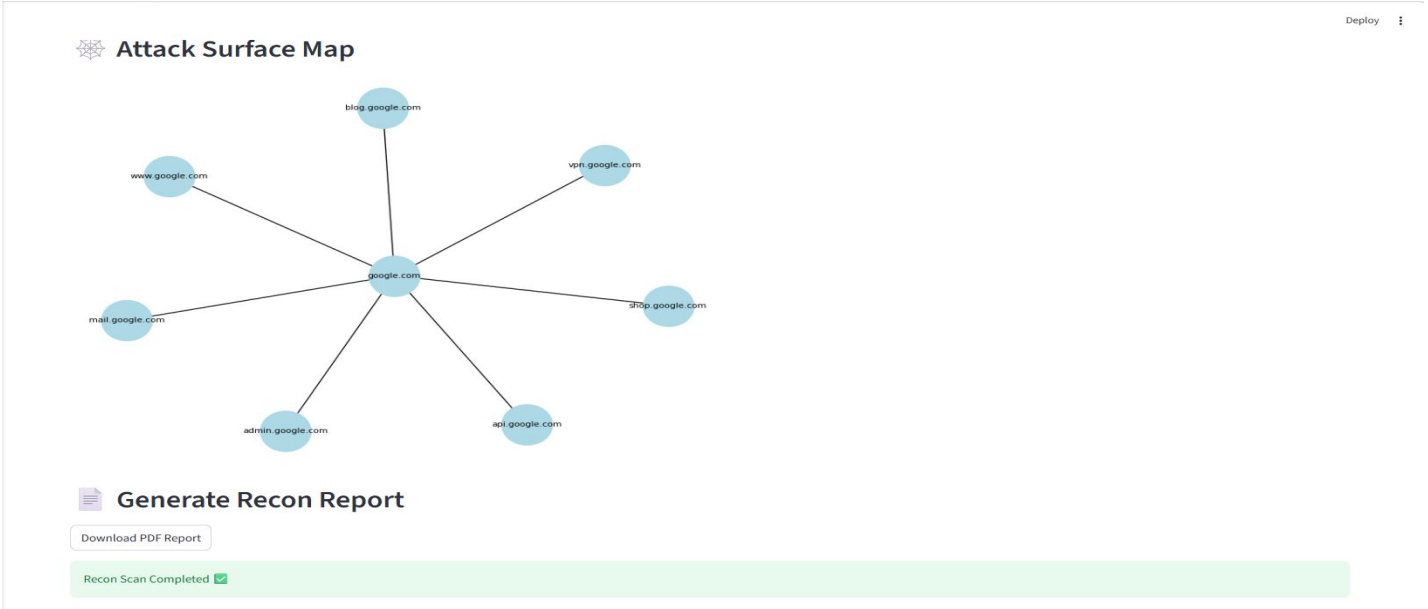
The system depends on **external reconnaissance tools**, so scanning speed depends on those tools.

Large-scale scans may take longer to complete.

AI analysis currently provides **basic vulnerability insights** and may require more advanced models for deeper analysis.

Outputs Screen





References

Nmap Official Documentation – Guide for network discovery and port scanning.
Retrieved from: https://nmap.org/docs.html

Subfinder Tool Documentation – ProjectDiscovery – Documentation for subdomain discovery tool used in reconnaissance.
Retrieved from: https://github.com/projectdiscovery/subfinder

WhatWeb Official Documentation – Tool used for identifying technologies used on websites.
Retrieved from: https://github.com/urbanadventurer/WhatWeb

Dirsearch GitHub Repository – Directory and file brute-forcing tool used for discovering hidden web directories.
Retrieved from: https://github.com/maurosoria/dirsearch

Python Official Documentation – Programming language used to develop and automate the reconnaissance system.
Retrieved from: https://docs.python.org/3/

Streamlit Documentation – Used for building the graphical interface and visualization dashboard for the project.
Retrieved from: https://docs.streamlit.io

Visual Studio Code Documentation – Development environment used for coding and managing the project.
Retrieved from: https://code.visualstudio.com/docs

Microsoft PowerShell Documentation – Used for running commands and executing tools in the Windows environment.
Retrieved from: https://learn.microsoft.com/en-us/powershell/

OpenAI ChatGPT – Used for assistance in understanding concepts, debugging code, and improving project documentation.
Retrieved from: https://chat.openai.com

Perplexity AI – Used as a research tool for gathering cybersecurity and reconnaissance-related information.
Retrieved from: https://www.perplexity.ai